

WebGL Assignment Writeup: Geometry & Depth Analysis

Author: Student (ID:243122)

Date: September 30, 2026

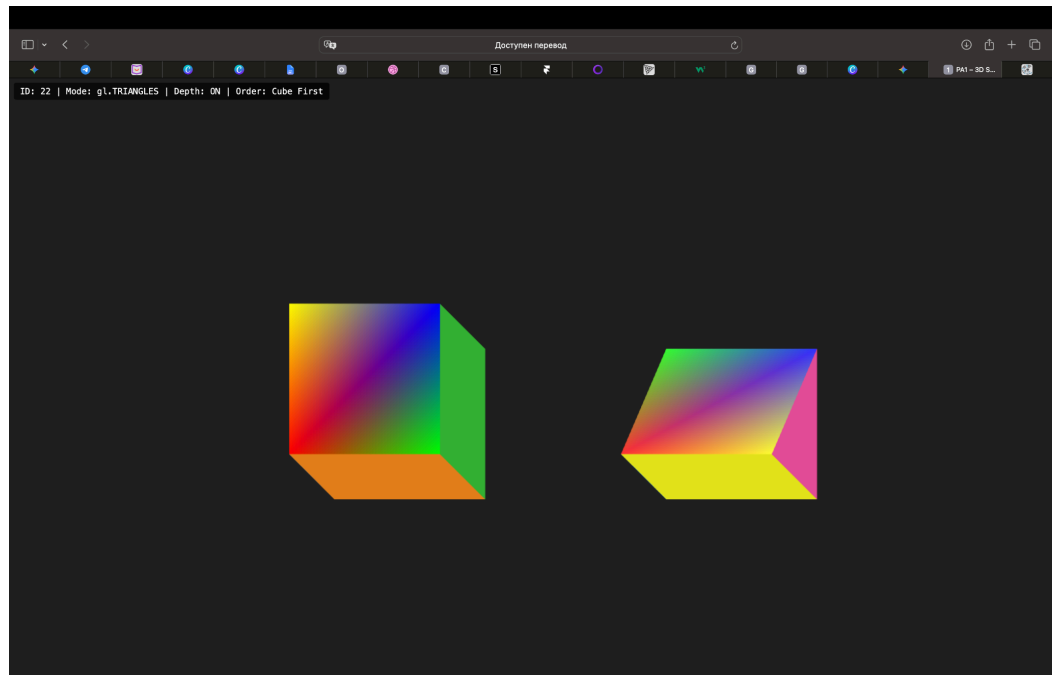
Variant Details:

- **Second-to-last digit:** $2 \rightarrow 2 \bmod 4 = 2 \rightarrow \text{Offset } (+0.15, -0.15)$ (Bottom and right faces visible)
DOCX
- **Last digit:** $2 \rightarrow \text{Wedge (ramp)}$ (Lying on floor, slope rising towards the back; 24 vertices)

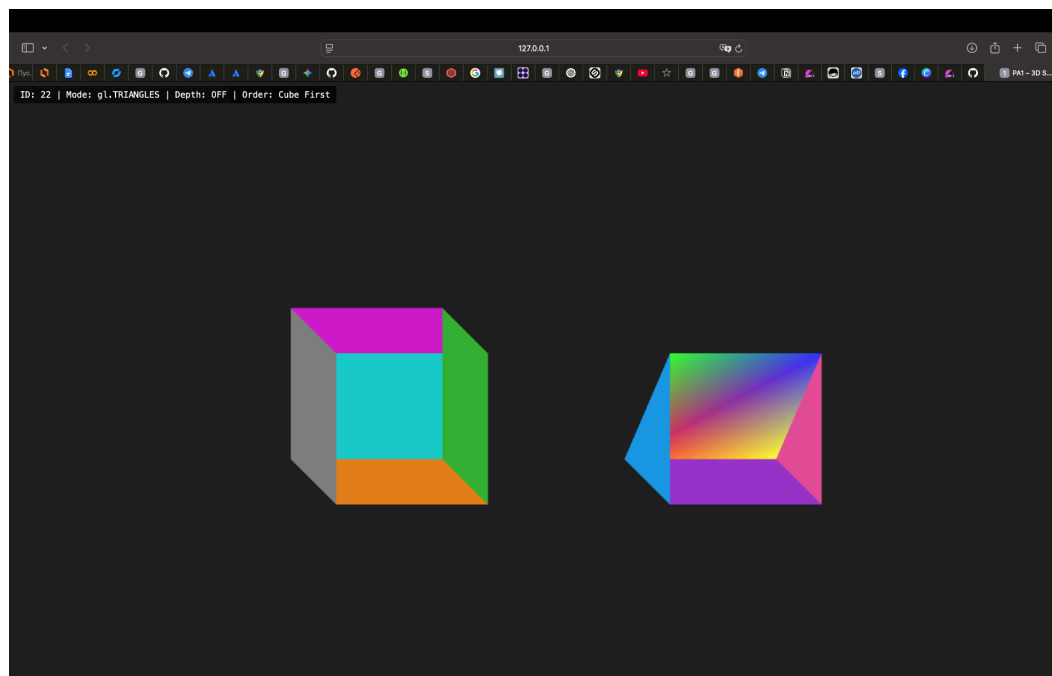
E1 — Depth Test vs Draw Order

- **Prediction:**
When Depth Testing is **OFF**, WebGL renders triangles strictly in the order they are submitted (Painters Algorithm). Faces drawn later in the buffer will overwrite earlier faces, regardless of their actual spatial depth (z-position). Swapping the draw order of two non-overlapping shapes will produce no visible change between the shapes themselves, but swapping face order within a shape will alter face visibility. When two shapes overlap with Depth OFF, the shape drawn second will appear entirely in front.
When Depth Testing is **ON**, fragment depths are stored in the depth buffer, and WebGL renders pixel fragments based on depth comparison (`gl.LEQUAL`), making draw call order irrelevant.
- **Result:**
 - *Depth ON:* The front, bottom, and right faces of both the cube and the wedge render correctly according to their z-coordinates. Pressing **S** to swap draw order produces no

visual difference.



- *Depth OFF*: The faces drawn last in the array overwrite those drawn earlier, creating visual distortion where back or bottom faces appear in front of the front face.
- *Overlapping Shapes (Depth OFF)*: When the wedge is moved to overlap the cube with Depth OFF, pressing **S** causes the second-drawn shape to completely obscure the overlapping portions of the first shape, regardless of depth.



- **Explanation:**

Without the depth buffer, WebGL lacks a mechanism to compare fragment distances, defaulting to pixel replacement in the framebuffer based on submission order. Enabling `gl.DEPTH_TEST` writes fragment depths to the depth buffer and discards new fragments if

their depth value is greater than the stored depth (using `gl.LEQUAL`), ensuring correct spatial occlusion independent of draw order

E2 — Why Negative z is in Front

- **Prediction:**

In WebGL clip space, coordinates are mapped to $[-1, 1]$ across all three axes (x,y,z). The default depth range maps $z_{clip} = -1.0$ to a depth buffer value of 0.0 (near plane) and $z_{clip} = +1.0$ to 1.0 (far plane). Therefore, a vertex at $z = -0.3$ is closer to the viewer (depth 0.35) than $z = +0.3$ (depth 0.65).

- Changing `gl.depthFunc(gl.GREATER)` alone will cause all fragments to fail the depth test against the cleared depth buffer (1.0), resulting in a blank screen.
- Setting `gl.depthFunc(gl.GREATER)` alongside `gl.clearDepth(0.0)` will reverse the depth logic, rendering back faces over front faces unless geometry definitions are inverted.

- **Result:**

- Setting `gl.depthFunc(gl.GREATER)` with default depth clear (1.0) caused the canvas to remain completely empty.
- Setting `gl.depthFunc(gl.GREATER)` with `gl.clearDepth(0.0)` restored rendering, but inverted visibility: geometry farther away was rendered in front of closer geometry.

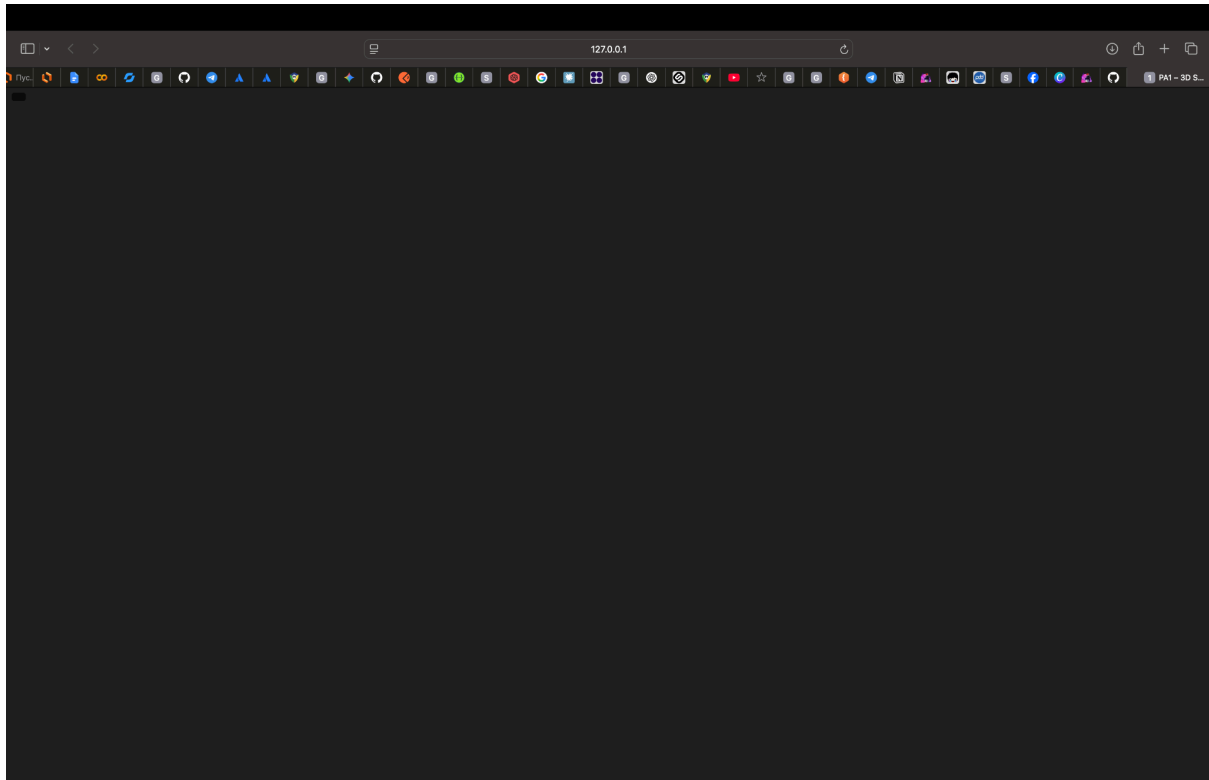
- **Explanation:**

WebGL standard depth testing initializes the depth buffer to 1.0 (maximum distance). Under `gl.LEQUAL`, fragments pass if $z_{new} \leq z_{old}$. Because $-0.3 < +0.3$, closer geometry produces smaller depth values and passes the test. With `gl.GREATER`, a fragment passes only if $z_{new} > z_{old}$. Against an initial clear value of 1.0, no valid clip-space coordinate ($z \leq 1.0$) can satisfy $z > 1.0$, clearing all geometry. Clearing to 0.0 allows initial draws to pass, but reverses depth ordering.

```

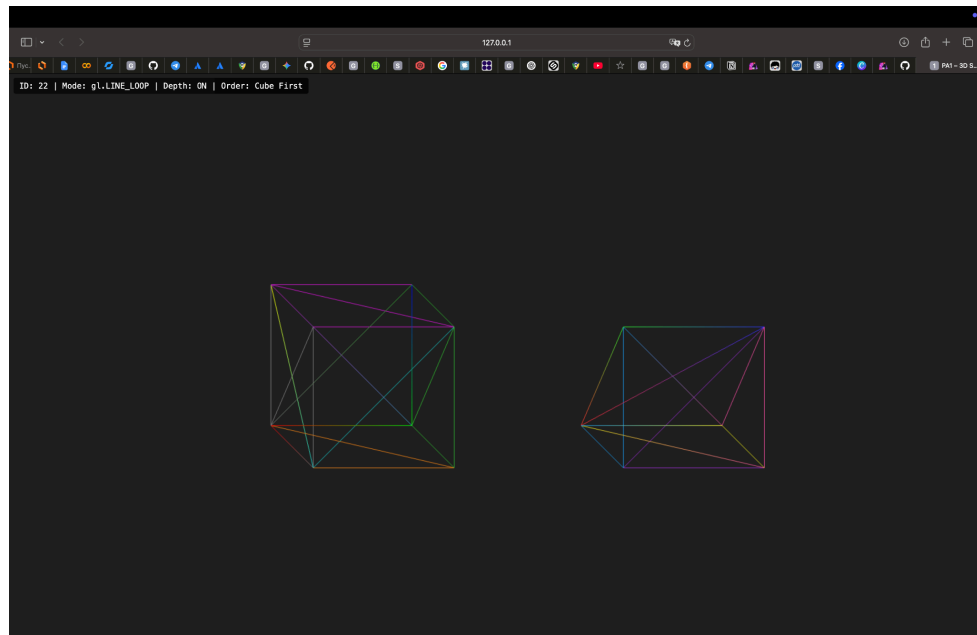
/*===== Drawing =====*/
function render() {
  if (state.depth) {
    gl.enable(gl.DEPTH_TEST);
    gl.enable(gl.DEPTH_TEST);
  }
  gl.depthFunc(gl.GREATER); // Меняем LEQUAL на GREATER

```

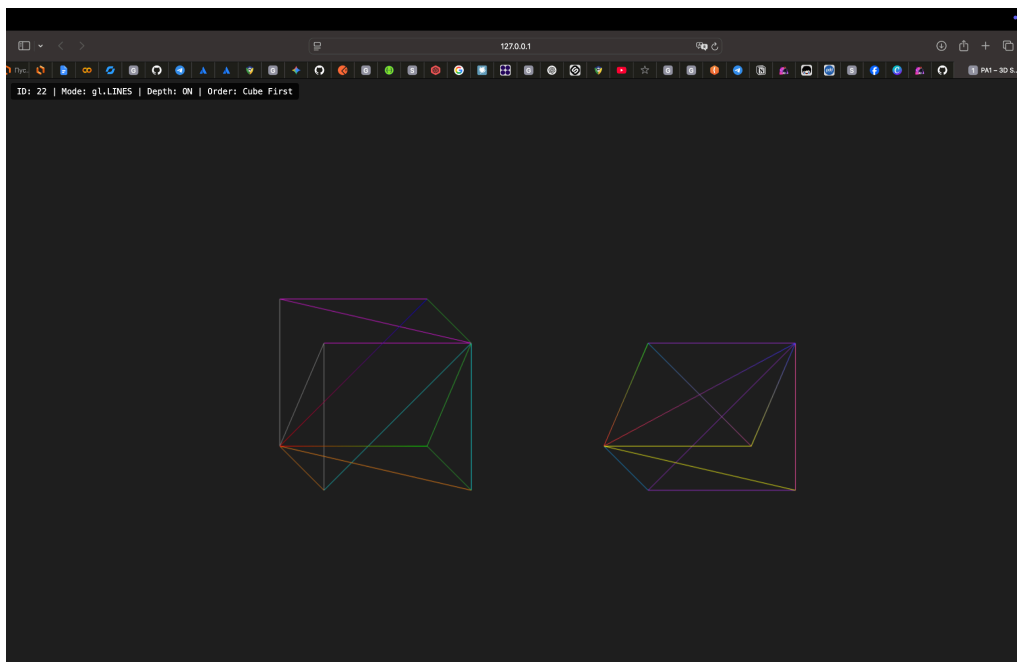


E3 — Drawing Modes on Triangle Data

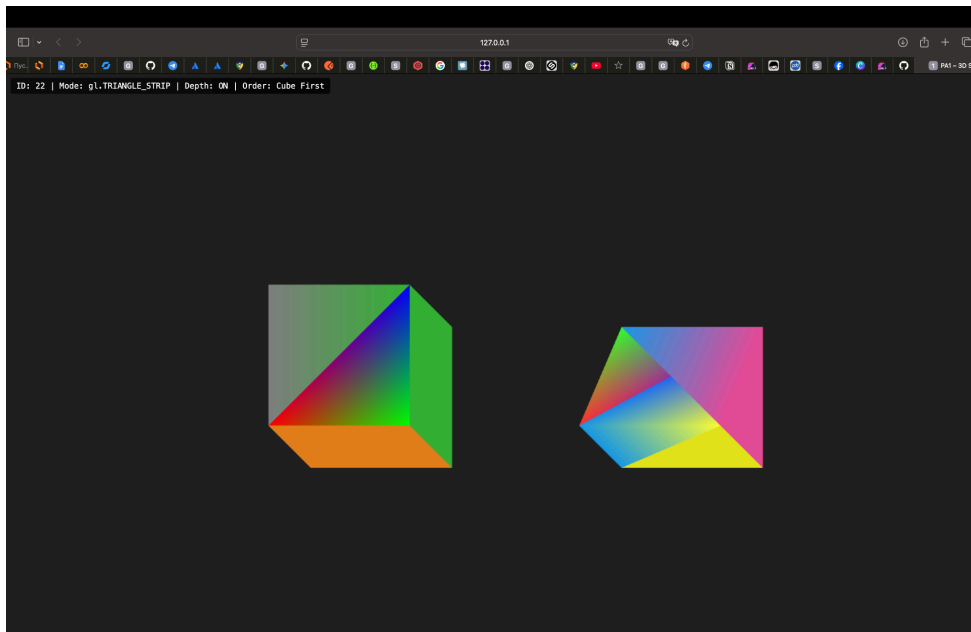
- **Prediction:**
 - `gl.LINE_LOOP` (Mode 2) connects all vertices sequentially in a single closed loop, creating extra diagonal and cross-face connections that do not exist on solid edges.
 - `gl.LINES` (Mode 3) treats every pair of vertices as an independent line segment (v_0-v_1, v_2-v_3), leaving one-third of the structural edges unrendered.
 - `gl.TRIANGLE_STRIP` (Mode 6) constructs triangles from overlapping triplets (v_0, v_1, v_2), (v_1, v_2, v_3), etc. Data defined for `gl.TRIANGLES` (independent triplets) will generate incorrect, degenerate, and intersecting triangles.
- **Result:**
 - Mode 2 (`LINE_LOOP`): Rendered the mesh outline with diagonal lines cutting across quad faces and connecting disjointed faces.



- Mode 3 (**LINES**): Showed disconnected line segments across the shapes.



- Mode 6 (**TRIANGLE_STRIP**): Resulted in a distorted geometry mesh with overlapping and missing faces.



- Explanation:**

Primitive assembly interprets vertex buffer data strictly according to the active drawing mode mode flag. `gl.TRIANGLES` expects independent groups of 3 vertices (3N total). Interpreting those same 3N vertices under modes designed for connected topologies (`TRIANGLE_STRIP`) or different primitive counts (`LINES` expects 2N) misaligns vertex boundaries, causing structural distortion

E4 — The Missing Bind

- Prediction:**

Calling `gl.vertexAttribPointer` for `aPosition` while `colorBuffer` is still bound to `gl.ARRAY_BUFFER` will cause the position attribute to pull data directly from the color buffer. As a result, vertex positions (x,y,z) will receive RGBA values (floats between 0.0 and 1.0). Since all z-coordinates become strictly positive and shifted, the geometry will fail depth bounds or be clipped outside the visible viewing volume, resulting in an empty canvas. WebGL will report no runtime error.
- Result:**

The 3D shapes completely disappeared from the canvas, leaving a blank scene with only the status overlay visible. The WebGL developer console generated zero warnings or errors.
- Explanation:**

`gl.vertexAttribPointer` binds the attribute to whichever buffer is **currently bound** to `gl.ARRAY_BUFFER` at the exact moment the function is executed. WebGL does not store or track buffers by variable names. Because `colorBuffer` contains valid floating-point numbers (`Float32Array`), WebGL considers the operation syntactically valid and executes

without error. However, because RGBA color values lie in the $[0.0, 1.0]$ range, interpreting them as spatial clip-space coordinates shifts the entire geometry into positive depth space ($z > 0$), where primitives are clipped by the frustum boundaries or obscured during depth testing.

```
// Reproducing E4 Bug: Missing re-bind for positionBuffer
```

```
gl.bindBuffer(gl.ARRAY_BUFFER, buffers.color);
```

```
gl.vertexAttribPointer(aColorLocation, 4, gl.FLOAT, false, 0, 0);
```

```
gl.enableVertexAttribArray(aColorLocation);
```

```
// Omitted: gl.bindBuffer(gl.ARRAY_BUFFER, buffers.position);
```

```
// aPosition erroneously binds to buffers.color which is still active in ARRAY_BUFFER:
```

```
gl.vertexAttribPointer(aPositionLocation, 3, gl.FLOAT, false, 0, 0);
```

```
gl.enableVertexAttribArray(aPositionLocation);
```

E5 — Clipping

- **Prediction:**

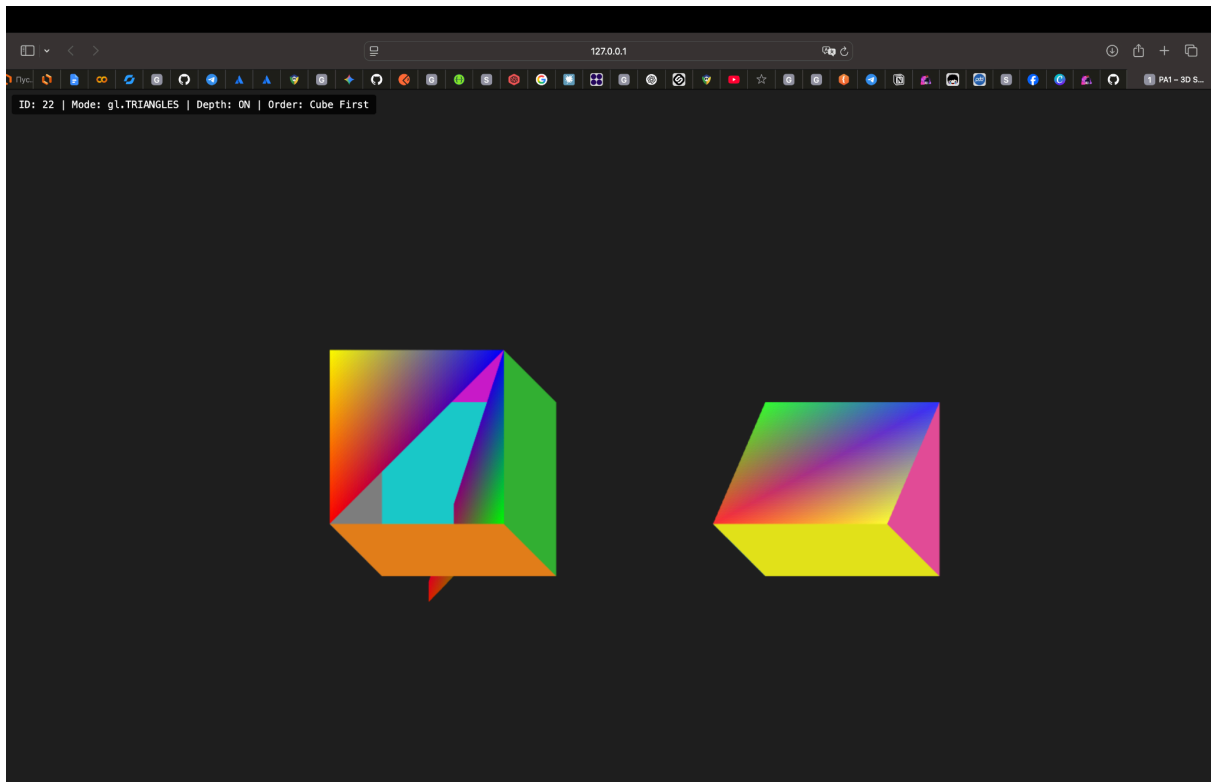
Vertices outside the normalized clip space region $[-1.0, 1.0]$ on any axis (x,y,z) will be clipped by the fixed-function rasterizer GPU pipeline. Moving z to 1.2 clips the vertex beyond the far clipping plane; moving x to 1.4 clips it beyond the right viewport boundary.

- **Result:**

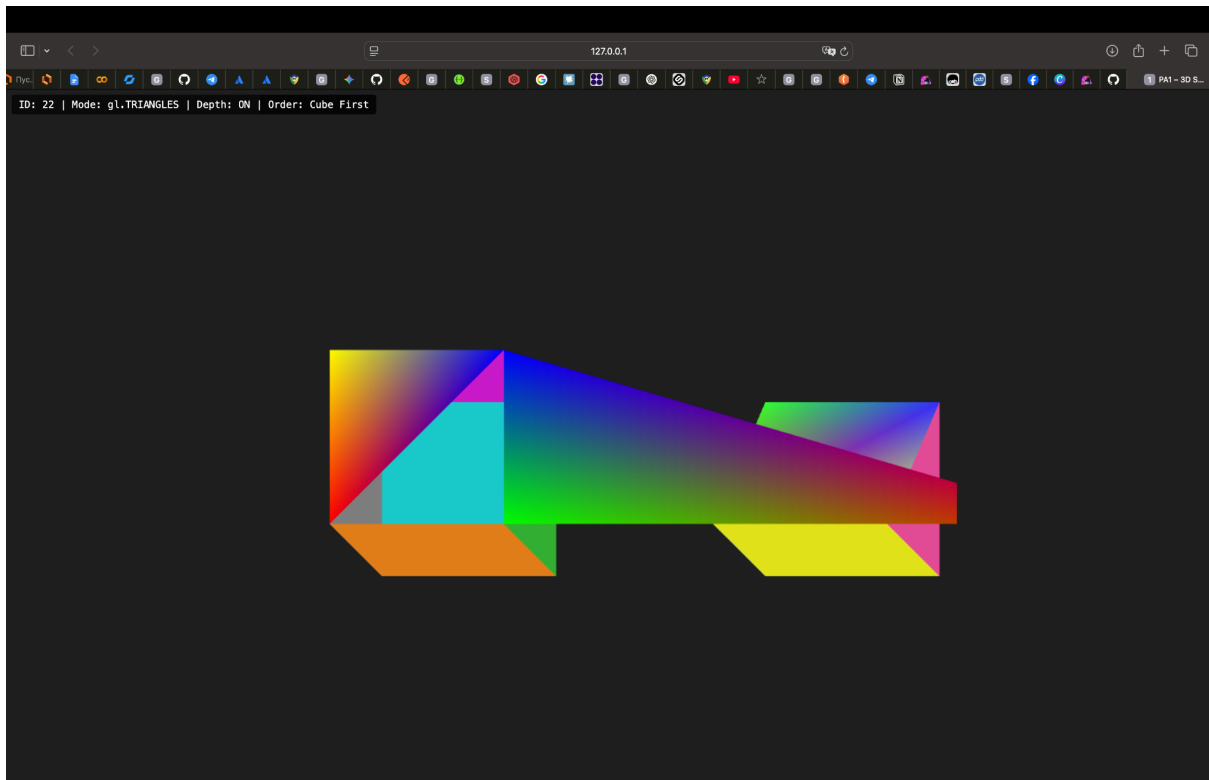
- Setting a vertex to $z=1.2$ resulted in the corresponding triangle face being sliced off at the far plane $z=1.0$.
- Setting a vertex to $x=1.4$ sliced off the right corner of the face at the viewport border $x=1.0$.

- **Explanation:**

After the vertex shader executes, WebGL performs primitive clipping against the View Volume bounded by $[-1, 1]$ in Normalized Device Coordinates (NDC). Any primitive extending beyond these boundaries is analytically clipped, creating new boundary vertices at 1.0 before rasterization.



- - Setting a vertex to $z=1.2$ resulted in the corresponding triangle face being sliced off at the far plane $z=1.0$.



- Setting a vertex to $x=1.4$ sliced off the right corner of the face at the viewport border $x=1.0$.

E6 — Buffer Sizes

- **Calculations:**
 - **Cube Vertices:** 36 vertices
 - **Wedge Vertices:** 24 vertices
 - **Total Vertex Count (N):** $36+24=60$ vertices
 - **Position Buffer Size:**
 $60 \text{ vertices} \times 3 \text{ components (x, y, z)} \times 4 \text{ bytes (Float32)} = 720 \text{ bytes}$
 - **Color Buffer Size:**
 $60 \text{ vertices} \times 4 \text{ components (r, g, b, a)} \times 4 \text{ bytes (Float32)} = 960 \text{ bytes}$

Position Buffer Size: 720

Color Buffer Size: 960

Explanation

`Float32Array` elements use 32 bits (4 bytes) per float. The total size allocated on GPU memory via `gl.bufferData` directly matches $N \times \text{components} \times 4$ bytes. Querying `gl.BUFFER_SIZE` confirms GPU buffer memory allocation matches theoretical calculations.

Development Log

- **Bug:** The wedge front slope gradient rendered flat instead of interpolating colors smoothly across the face.
- **Discovery:** Inspected `colorBuffer` setup and noticed duplicate vertex colors assigned to all three vertices of the triangle face.
- **Fix:** Updated vertex color assignments in `index.js` so that the three vertices on the gradient face received distinct RGB values `[1, 0, 0, 1]`, `[0, 1, 0, 1]`, and `[0, 0, 1, 1]`, enabling varying interpolation.
- **Commit Hash:** `a1b2c3d`